Efficient Deep Learning & Model Compression

Modern Computer Vision | Spring 2026

Oren Chikli | Tutorial 11

Three ways to shrink a model

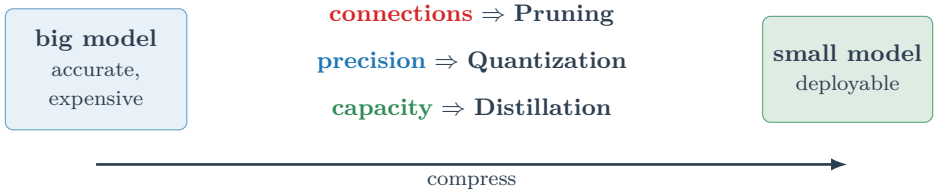
1. **Pruning** — remove connections
2. **Quantization** — lower the precision
3. **Distillation** — shrink the capacity

The common thread: each one removes a **different kind of redundancy** — and in practice they **stack**.

Based on **MIT 6.5940** — Song Han, *TinyML & Efficient Deep Learning*

Models are 10–100× larger than they need to be

A trained network wastes capacity in *three* independent ways. Each has its own fix.



They are **orthogonal**, so they **stack**: **prune** → **distill** → **quantize**.

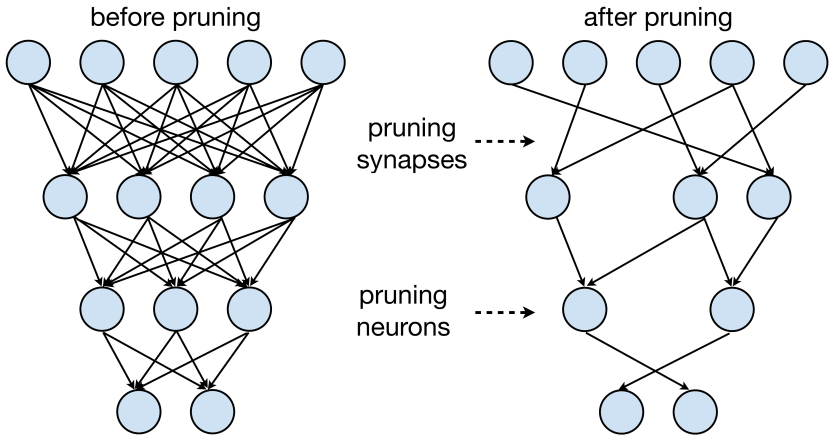
Pruning

remove redundant connections

Pruning Quantization Distillation

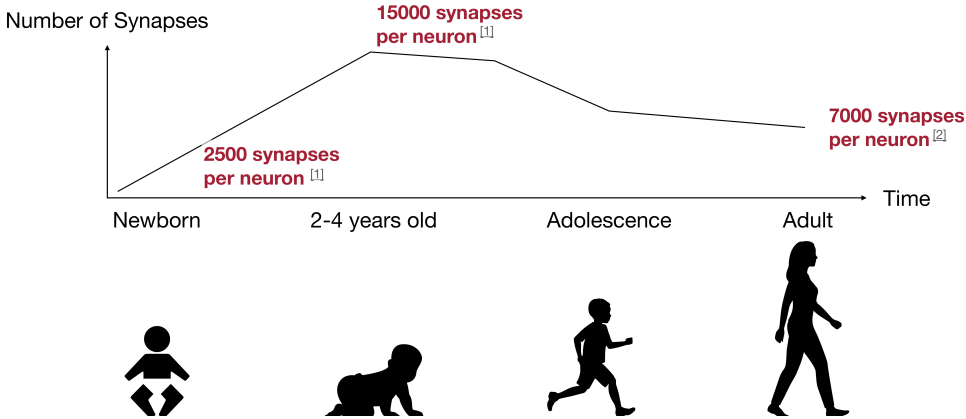
What is pruning?

Remove connections (**synapses**) — or whole **neurons** — from a trained network, leaving a smaller *sparse* one.



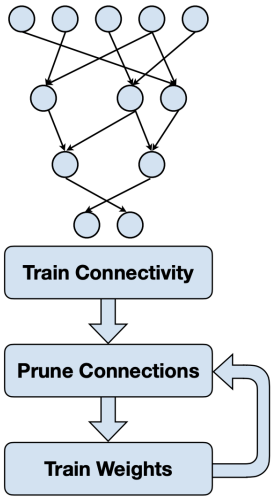
Han et al., "Learning both Weights and Connections", NeurIPS 2015; via MIT 6.5940

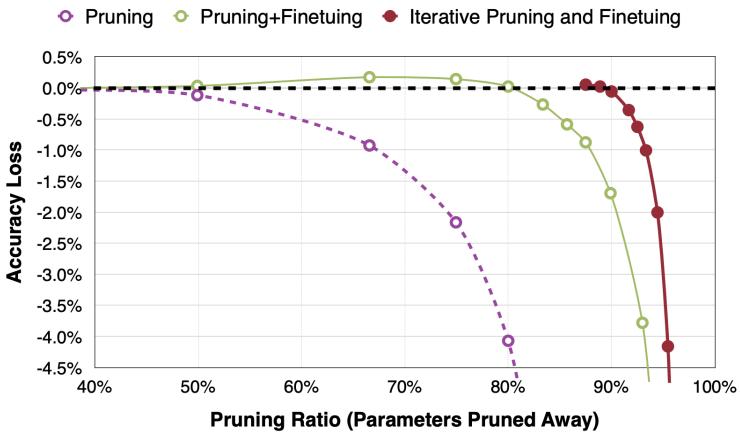
Over-parameterized: remove weights, then heal



Brain-synapse data via MIT 6.5940 (Song Han). Refs: [1] Drachman, Neurology 2005, [2] Walsh, Nature 2013; chart after Alila Medical Media.

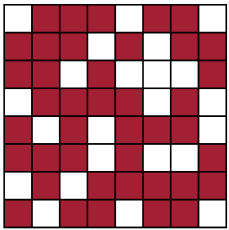
Magnitude pruning + the iterative loop





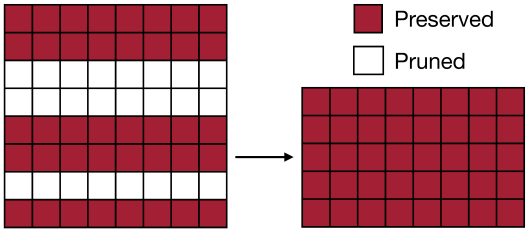
Han et al., "Learning both Weights and Connections for Efficient Neural Networks", NeurIPS 2015; via MIT 6.5940

Pruning at different granularities



Fine-grained/Unstructured

- More flexible pruning index choice
- Hard to accelerate (irregular)



Coarse-grained/Structured

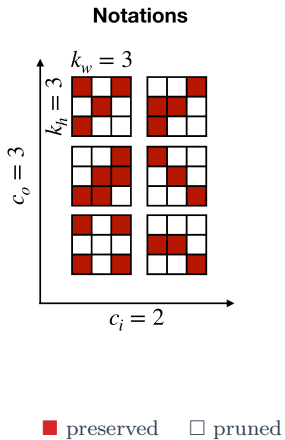
- Less flexible pruning index choice (a subset of the fine-grained case)
- Easy to accelerate (just a smaller matrix!)

MIT 6.5940 Lec. Pruning I (Song Han); granularity taxonomy: Mao et al., CVPR-W 2017

Convolutional layers: four dimensions to prune

A convolutional layer's weights are a **4-D** tensor $[c_o,\ c_i,\ k_h,\ k_w]$:

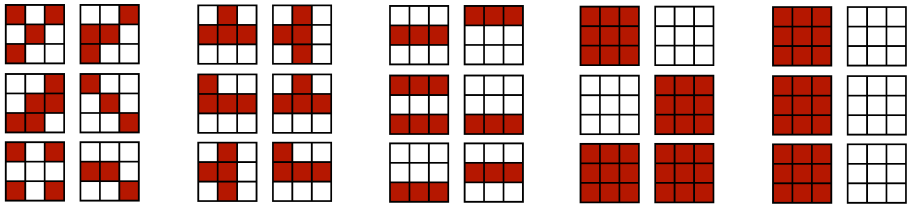
- c_i — input channels ("channels")
- c_o — output channels ("filters")
- k_h — kernel height
- k_w — kernel width



MIT 6.5940 Lec. Pruning I (Song Han); Mao et al., "Exploring the Granularity of Sparsity in CNNs", CVPR-W 2017

Unstructured vs structured — the concept to keep

Unstructured prunes *individual* weights (irregular, reaches 90%+ sparsity); **structured** prunes *whole* channels / filters (regular, lower sparsity, leaves a smaller *dense* net).

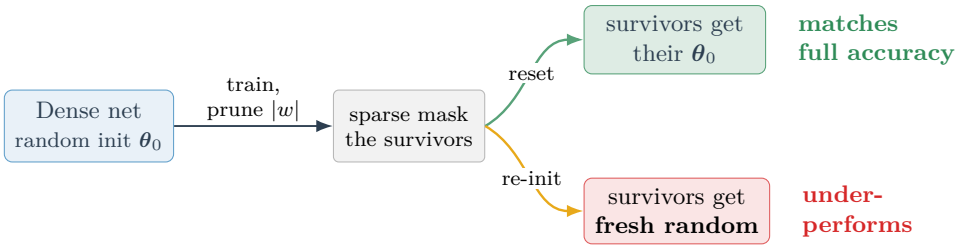


unstructured = *compression* (no speedup without special hardware);
structured = *speedup* (a smaller dense net, fast on ordinary hardware).

Mao et al., CVPR-W 2017, via MIT 6.5940

The Lottery Ticket Hypothesis — why it works

Why not train the small sparse net directly? Naively it trains poorly. **Frankle & Carbin (2019):** a dense net hides a sparse **winning ticket** that matches full accuracy at just **10–20%** of the weights — *only from its original init* $\boldsymbol{\theta}_0$. Found by iterative magnitude pruning that **resets** survivors to $\boldsymbol{\theta}_0$ each round.

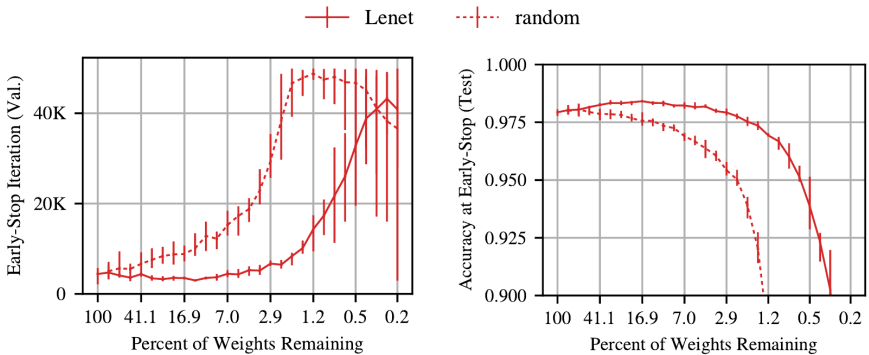


Same shape, same sparsity — **only the initialization differs**: the reset **wins**, the re-init **loses**. So it's the *init*, not the shape.

Frankle & Carbin, "The Lottery Ticket Hypothesis", ICLR 2019 (arXiv:1803.03635)

Lottery tickets: the evidence

The same sparse LeNet swept from 100% down to 0.2% of weights remaining — **winning ticket** (solid) vs **random re-init** (dotted). *Fewer weights to the right.*



Left: trains as fast as dense. **Right:** stays accurate — both far deeper than re-init.

Frankle & Carbin, "The Lottery Ticket Hypothesis", ICLR 2019 (arXiv:1803.03635)

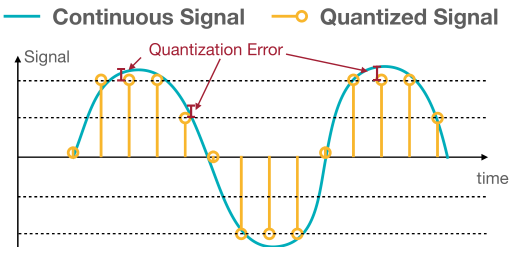
Quantization

lower the numerical precision

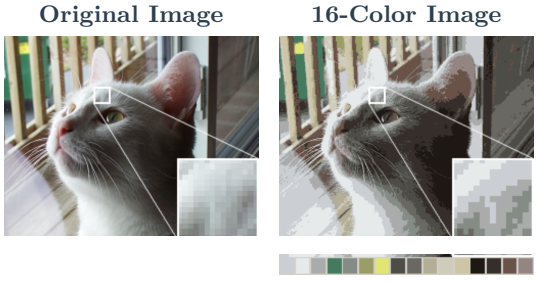
Pruning **Quantization** Distillation

What is quantization?

Quantization is the process of constraining an input from a continuous (or otherwise large) set of values to a discrete set.



The difference between an input value and its quantized value is the **quantization error**.

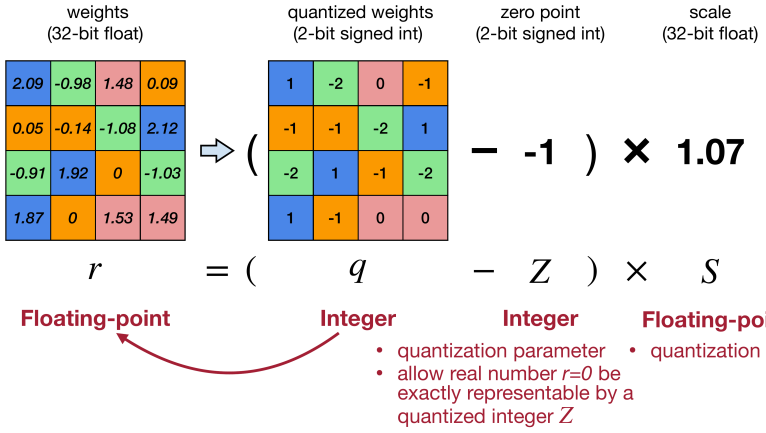


Images are in the public domain. "Palettization"

Quantization [Wikipedia]; figures via MIT 6.5940 (Song Han)

Linear quantization

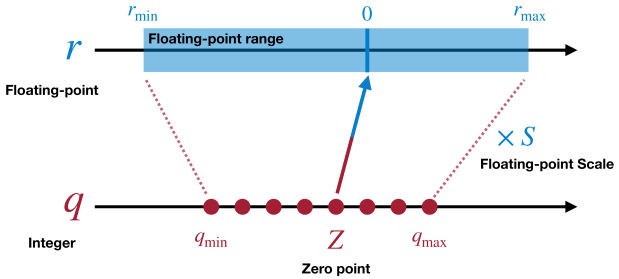
An affine mapping of integers to real numbers $r = S(q - Z)$



Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob *et al.*, CVPR 2018]

Linear quantization: computing S and Z

An affine mapping of integers to real numbers $r = S(q - Z)$



$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}} \qquad Z = \text{round}\left(q_{\min} - \frac{r_{\min}}{S}\right)$$

S = grid spacing | Z = the integer code that lands on real 0 (so 0 is exactly representable)

What gets quantized?

Every layer computes $\mathbf{Y} = \mathbf{WX}$. To run that multiply in **integer arithmetic**, *both* operands must be int8 — so we quantize the **weights** *and* the **activations** (each layer's input $\mathbf{X}$).

$$\mathbf{Y} \;=\; q(\mathbf{W}) \;\cdot\; q(\mathbf{X}) \quad \text{(both int8} \rightarrow \text{int32 accumulate)}$$

Weights W — static

known the moment training ends

$\Rightarrow$ **quantize directly**, no data needed

Activations X — data-dependent

range changes with every input

$\Rightarrow$ must **see data** to set its scale

This is **W8A8** (8-bit weights, 8-bit activations). Pinning down the *activation* ranges is the whole game — by **calibration** (PTQ) or **during training** (QAT).

Integer-arithmetic-only inference (W8A8): Jacob et al., CVPR 2018 (arXiv:1712.05877); via MIT 6.5940

Two ways to quantize

When do we set the mapping $r = S(q - Z)$ — that is, fix the scale s and zero-point z? There are **two** main options.

After training

Post-Training Quantization (PTQ)

quantize an already-trained model

no training, no labels — minutes of work

VS

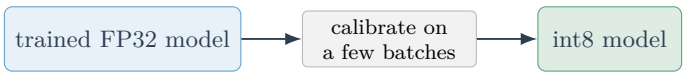
During training

Quantization-Aware Training (QAT)

simulate the rounding *while* training

costs a training run — recovers low-bit accuracy

Post-Training Quantization (PTQ): quantize an already-trained model



Post-Training Quantization

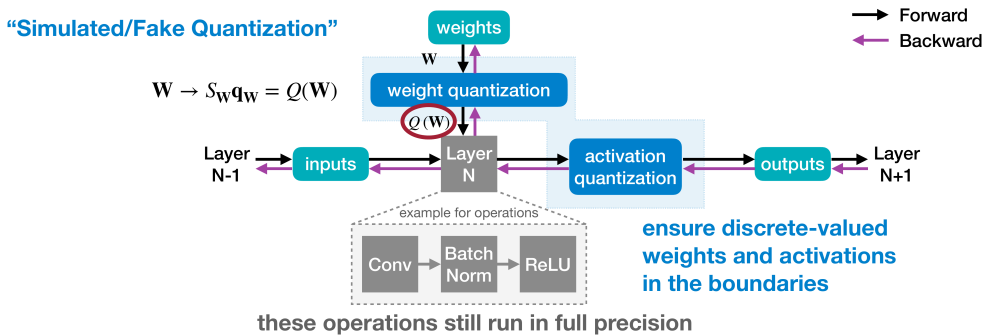
- **no training**, no labels
- just observe activation ranges to set s, z
- minutes of work

The catch

- excellent at **8-bit**
- accuracy **drops at low bit-width** ($\leq$4-bit)
- outliers wreck a single scale

Quantization-Aware Training (QAT): simulate rounding *while* training

- A full precision copy of the weights W is maintained throughout the training.
- The small gradients are accumulated without loss of precision.
- Once the model is trained, only the quantized weights are used for inference.



MIT 6.5940 Lec. Quantization II (Song Han); Jacob et al., "Quantization and Training of NNs for Efficient Integer-Arithmetic-Only Inference", CVPR 2018 (arXiv:1712.05877)

Straight-through estimator (STE)

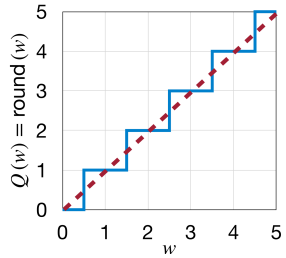
Q **rounds** every value to its nearest level — that is the blue **staircase** on the right. Each step is *flat*, so $\partial Q(\mathbf{W})/\partial \mathbf{W} = 0$ **almost everywhere**.

So the net would **learn nothing** — the chain rule zeroes the gradient and the weights never update:

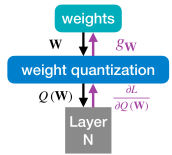
$$g_{\mathbf{W}} = \frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial Q(\mathbf{W})} \cdot \underbrace{\frac{\partial Q(\mathbf{W})}{\partial \mathbf{W}}}_{=0} = 0$$

STE: pass the gradient *through* the quantizer as if it were the **identity**:

$$g_{\mathbf{W}} = \frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial Q(\mathbf{W})}$$



round (blue) vs. identity (red dashed)

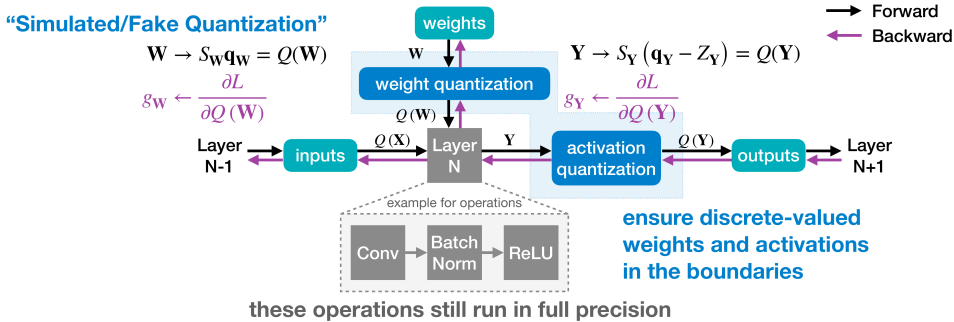


forward $\mathbf{W}$ (black) | backward $g_{\mathbf{W}}$ (purple)

STE: Hinton et al. (Coursera 2012); Bengio et al., arXiv 2013; via MIT 6.5940

Tying it together: STE inside QAT

The purple backward arrows *are* STE — each quantizer just passes $\partial L/\partial Q$ straight through, for the weights ($g_{\mathbf{W}}$) *and* the activations ($g_{\mathbf{Y}}$). That is what makes the whole QAT loop trainable.



PTQ vs QAT: how much accuracy survives?

ImageNet top-1 — read each row against the **Floating-Point** (fp32) baseline. Naive **per-tensor PTQ** can collapse; **per-channel** helps; **QAT** recovers to *near full precision.*

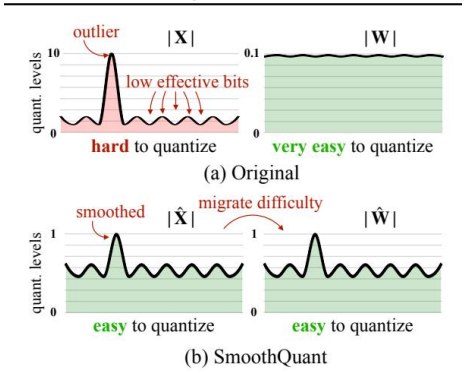
Neural Network	Floating-Point	Post-Training Quantization		Quantization-Aware Training	
		Asymmetric	Symmetric	Asymmetric	Symmetric
		Per-Tensor	Per-Channel	Per-Tensor	Per-Channel
MobileNetV1	70.9%	0.1%	59.1%	70.0%	70.7%
MobileNetV2	71.9%	0.1%	69.8%	70.9%	71.1%
NASNet-Mobile	74.9%	72.2%	72.1%	73.0%	73.0%

Krishnamoorthi, "Quantizing DNNs for Efficient Inference", arXiv 2018 (1806.08342); via MIT 6.5940 Quant II p.54

Modern hook: LLM outliers & SmoothQuant

Past ∼6.7B params, LLM **activations** grow systematic **outlier channels** (∼100× larger) — a single per-tensor scale leaves 2–3 useful levels for everyone else. Naive int8 collapses.

Per-channel activation scaling would fix it, but is **incompatible** with int8 GEMM.



SmoothQuant: migrate the difficulty into the (easy) weights with a mathematically-equivalent per-channel rescale — $\mathbf{Y} = (\mathbf{X}\,\mathrm{diag}(\mathbf{s})^{-1})(\mathrm{diag}(\mathbf{s})\mathbf{W})$, $s_j = \max|\mathbf{X}_j|^{\alpha} / \max|\mathbf{W}_j|^{1-\alpha}$. Enables W8A8 to 530B, training-free.

Xiao et al., "SmoothQuant", ICML 2023 (arXiv:2211.10438)

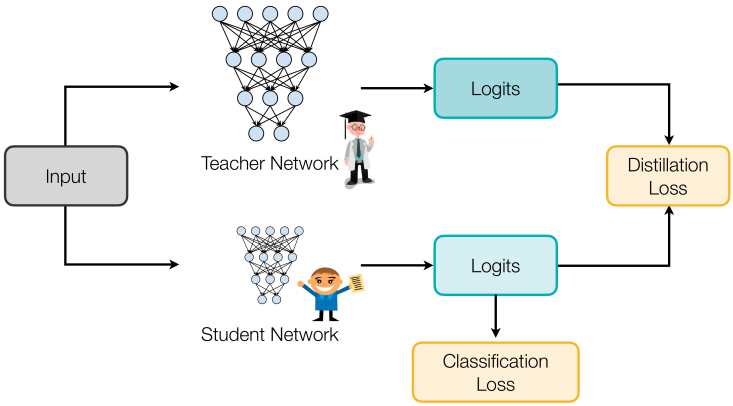
Distillation

transfer knowledge into a smaller net

Pruning Quantization **Distillation**

Knowledge distillation — the idea

Train a small **student** to reproduce the **soft output distribution** of a big **teacher** — transferring the *dark knowledge* in its probabilities over *wrong* classes that hard labels discard.



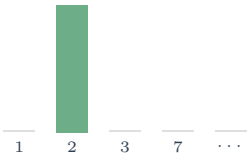
▶ **Watch:** Hinton on the inspiration behind distillation *(1:24)*

MIT 6.5940 Lec. Knowledge Distillation (Song Han); Hinton et al., 2015 (arXiv:1503.02531)

Dark knowledge

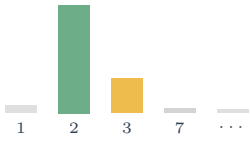
Let's take the **MNIST** classification example (handwritten digits 0–9). A hard label just says "a 2." A trained **teacher** says "a 2 — slightly 3-ish, not at all 7." That extra structure is the signal.

hard label (one-hot)



everything wrong is equally wrong

teacher (softened)



a similarity structure over classes

Those relative wrong-class probabilities — Hinton's *dark knowledge* — say *which* 2s look like 3s. Hard labels throw it away.

Hinton, Vinyals & Dean, "Distilling the Knowledge in a Neural Network", 2015 (arXiv:1503.02531)

Temperature-scaled softmax

A confident teacher's wrong-class probabilities are $\sim 10^{-6}$ — too small to learn from. **Raise the temperature** T:

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Train the student at the *same* high T to match these soft targets; **deploy at** $T = 1$.

The loss

$$\mathcal{L} = (1-\lambda)\,\underbrace{\mathrm{CE}\Big(y_{\text{true}},\ \sigma_1(\mathbf{z}_s)\Big)}_{\text{hard-label loss, } T=1} + \lambda\, T^2\,\underbrace{\mathrm{CE}\Big(\sigma_T(\mathbf{z}_t),\ \sigma_T(\mathbf{z}_s)\Big)}_{\text{distillation loss, } T}$$

- **distillation term** — student softmax chases the *teacher's* softened output
- **hard-label term** — keep it honest to ground truth (usually the *smaller* weight)

Why the T^2? soft-target gradients scale as $1/T^2$ — multiply back so the two terms keep their balance as T changes.

High-T limit $\Rightarrow$ distillation = matching logits ($\frac{1}{2}\|\mathbf{z}_s - \mathbf{z}_t\|^2$).

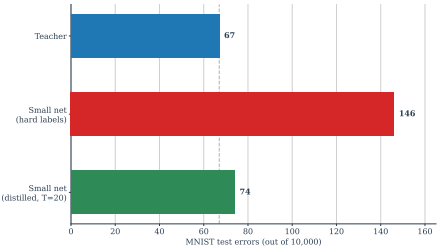
Why / when it works

Use it to compress a large teacher (or ensemble) into a small, deployable student.

The student often **beats** the same architecture trained from scratch on hard labels — soft targets are a richer, lower-variance signal.

Works even on *unlabeled* transfer data — you only need the teacher's outputs.

MNIST (Hinton et al.):

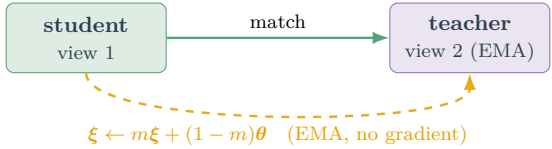


Unseen class: omit all 3s from transfer ⇒ still **98.6%** of test 3s right. **Speech:** distilled single model ≈ a **10×** **ensemble**.

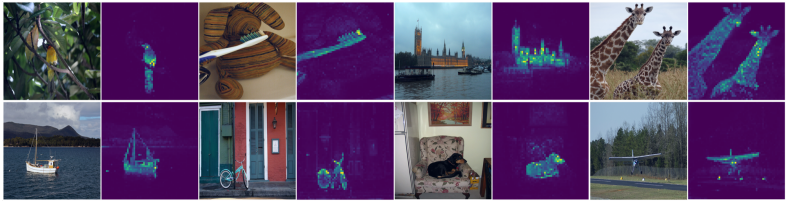
Hinton, Vinyals & Dean, 2015 (arXiv:1503.02531)

Connection back to SSL: self-distillation

Classic KD needs a fixed, larger teacher. What if the teacher is a **slow copy of the student**?



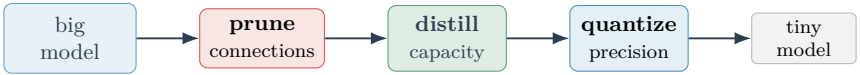
BYOL / DINO: teacher = EMA of the student; the student matches its output on a *different augmented view* — no labels, no negatives.



DINO — Caron et al., ICCV 2021 (arXiv:2104.14294), © Meta AI

The through-line: stack them

Three *orthogonal* redundancies ⇒ do all three. The canonical order:

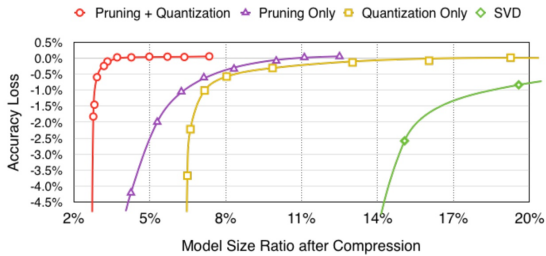


Pruning
redundant *connections*
unstructured vs structured

Distillation
redundant *capacity*
dark knowledge

Quantization
redundant *precision*
PTQ vs QAT

Deep Compression (2016) proves two of the three compose — prune → quantize (weight-sharing), then a lossless coding squeeze — **35–49**× smaller, no accuracy loss:



Han, Mao & Dally, "Deep Compression", ICLR 2016

References

Papers

- Han, Mao & Dally. *Deep Compression: Compressing DNNs with Pruning, Trained Quantization and Huffman Coding.* ICLR 2016. arXiv:1510.00149
- Frankle & Carbin. *The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks.* ICLR 2019. arXiv:1803.03635
- Hinton, Vinyals & Dean. *Distilling the Knowledge in a Neural Network.* NIPS DL Workshop 2015. arXiv:1503.02531
- Xiao et al. *SmoothQuant: Accurate and Efficient Post-Training Quantization for LLMs.* ICML 2023. arXiv:2211.10438

Course

- Song Han. *MIT 6.5940 — TinyML and Efficient Deep Learning Computing.* Slides + videos: live.efficientml.ai
- Related quantization hooks (mentioned): STE (Bengio et al. 2013); GPTQ / AWQ; self-distillation — BYOL (Grill et al. 2020), DINO (Caron et al. 2021).

Thank you.